

Python for Expert R Users: A Migration Path

September 2026

Table of Contents

1	Establish the migration foundation	2
1.1	Keep the analytical contract; change the implementation surface . . .	2
1.2	A deliberately boring default stack	3
1.3	The Python defaults that matter most	4
2	Port table work as named, testable transformations	5
2.1	Use pandas for an explicit first translation	5
2.2	A compact translation table	6
2.3	Test the properties that make an analytical result trustworthy . . .	7
3	Keep functional clarity, but use Python's native tools	7
3.1	Prefer straightforward functions and comprehensions	8
3.2	Make failure policy part of the function boundary	8
3.3	Treat resources and configuration as explicit dependencies	9
4	Separate inference, prediction, and visual communication	9
4.1	Choose a model stack by the question, not by package loyalty . . .	9
4.2	Rebuild chart meaning before chart styling	10
5	Improve performance in the order that preserves trust	11
5.1	Vectorise and reduce data movement first	12
5.2	Add parallelism only for independent, coarse work	12
6	One end-to-end case: migrate a monthly churn scorecard	13
6.1	Stage 1: freeze the reference outcome	13
6.2	Stage 2: port transformations before modelling	14
6.3	Stage 3: compare the model and chart at decision level	15
6.4	Stage 4: profile, package, and make the cutover reversible	15
7	Migrate as a team, with a recovery path	16
7.1	Put shared agreements in the repository	16
7.2	Recognise the recurring anti-patterns	16
7.3	A concise recovery sequence	17
7.4	Further Reading	18

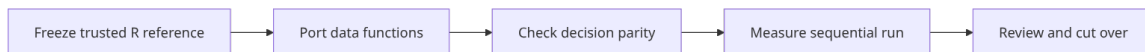
Moving from R to Python is not a request to abandon disciplined analysis. It is a request to make more of that discipline explicit: types, object boundaries, interfaces, tests, execution limits, and ownership. A line-by-line port usually preserves neither the clarity of a good R workflow nor the operational strengths of Python. This primer gives you a route for moving one trusted workflow safely.

This is for experienced R users: people already comfortable with tidyverse transformations, purrr, ggplot2, formula models, and production-ish analytical work. It assumes ordinary Python literacy is not yet the goal. You will learn a default Python path for table work, functions, models, plots, profiling, and team delivery. You will not get a language reference, an exhaustive package catalogue, or a claim that every specialised R package has a drop-in equivalent.

The core route is deliberately narrow:

1. Freeze a trusted R result and its data contract.
2. Port its table transformations as small, testable Python functions.
3. Reproduce the decision-relevant model and chart outputs.
4. Measure the sequential Python workflow before changing its execution model.
5. Cut over only after parity, review, and rollback are credible.

Everything else in this primer supports that route. Read it in order for a first migration. If you are already blocked, start with the relevant section, then return to the end-to-end case study in Section 6.



1 Establish the migration foundation

1.1 Keep the analytical contract; change the implementation surface

The useful part of an established R workflow is not its syntax. It is the contract it has accumulated: source tables, null policy, join cardinality, feature definitions, splits, metrics, chart semantics, runtime expectations, and people who can explain the result. Port that contract before you optimise anything.

Start with one workflow that is valuable, bounded, and already trusted. A monthly scorecard, recurring forecast, or classification report is better than

a programme to rewrite every notebook. Capture a fixed input extract, the R output tables, expected column types, acceptance tolerances, and a short explanation of how the output is used. Numerical parity is not always bit-for-bit parity: different libraries can vary in optimisation or floating-point order. Define the tolerance before looking at the new result.

Contract	Record before coding	Typical migration failure
Data shape	keys, grain, row-count range, nullable fields	an accidental many-to-many join doubles totals
Transformations	ordering, filters, missing-value rules	a NaN is treated as an ordinary value
Model	split date, features, encoding, threshold	the estimator matches but leakage changes the result
Visual output	measure, grouping, scales, annotations	a plausible chart answers a different question
Operations	runtime, memory, rerun/rollback path	a fast local script cannot run predictably in CI

1.2 A deliberately boring default stack

Use a small stack until evidence requires more. For most analytical migrations, that means pandas and numpy for tables and arrays; pytest for checks; matplotlib and seaborn for static plots; statsmodels for inference-oriented models; and scikit-learn for predictive pipelines. Use pathlib for paths and the standard library before reaching for a framework.

polars, dask, Ray, Numba, and specialised plotting libraries can be excellent tools. They are not the default answer to a first port. Introducing a new execution model, lazy query engine, or distributed runtime while also translating business logic makes parity failures needlessly hard to isolate.

Use a project-local environment and a lockable dependency declaration. The exact tool is a team choice; the important rule is that a colleague and CI can recreate the same environment. Keep the first port as importable modules plus a thin runnable entry point, rather than a notebook that owns production logic.

```

project/
├── src/scorecard/
│   ├── io.py           # reading and writing boundaries
│   ├── transforms.py   # deterministic table functions
│   ├── model.py        # fitting and scoring
│   └── report.py       # tables and charts
├── tests/
│   ├── test_transforms.py
│   └── test_parity.py
├── data_contract.md
└── run_scorecard.py

```

1.3 The Python defaults that matter most

R users often hit trouble not because Python lacks an equivalent verb, but because Python asks them to see state that R workflows often hide.

Object identity and mutation. Two Python names can point to the same object. Do not mutate a slice and hope the result is independent. Take a deliberate copy at a boundary, then use `.loc` for assignment.

```

active = orders.loc[orders["status"].eq("active"), ["customer_id", "amount"]].copy()
active.loc[:, "amount_share"] = active["amount"] / active["amount"].sum()

```

Nulls and types. Python has `None`, floating-point `NaN`, and pandas nullable values such as `pd.NA`. Decide which fields are nullable and use nullable dtypes where that matters. Never use `== None` or `== np.nan` as a missing-data test; use `.isna()`.

```

import pandas as pd

```

```

customers = customers.astype({
    "customer_id": "Int64",
    "segment": "string",
    "is_active": "boolean",
})

```

Index state. A pandas index is useful but it is not a primary key. In analytical migration work, keep business keys as ordinary columns and favour `groupby(..., as_index=False)` until you have a reason not to. That avoids a large class of merge and plotting surprises.

Shape and cardinality. Treat row counts and join relationships as assertions.

`validate="many_to_one"` turns a quiet data-quality error into a useful failure.

```

enriched = orders.merge(
    customers,
    on="customer_id",
    how="left",
    validate="many_to_one",
)
if enriched["segment"].isna().any():
    raise ValueError("Unmatched customer IDs in orders")

```

These habits look explicit because they are. In a long-lived workflow, that visibility is an advantage: review can focus on assumptions rather than guessing what a compact expression will do.

2 Port table work as named, testable transformations

2.1 Use pandas for an explicit first translation

The closest productive mental model is not “make pandas look like dplyr.” It is “turn each important pipeline stage into a function with an input and output contract.” Method chains are fine for a local operation; named functions are better where a reviewer needs to understand a business rule, an error boundary, or a reusable stage.

This R pipeline has three decisions: discard invalid rows, calculate a monthly customer grain, and attach customer attributes.

```
monthly <- transactions %>%  
  filter(!is.na(amount), amount >= 0) %>%  
  mutate(month = lubridate::floor_date(transaction_date, "month")) %>%  
  group_by(customer_id, month) %>%  
  summarise(  
    orders = n(),  
    revenue = sum(amount),  
    .groups = "drop"  
  ) %>%  
  left_join(customers, by = "customer_id")
```

The Python version should expose those same decisions, rather than trying to win a line-count contest.

```
import pandas as pd
```

```
def build_monthly_sales(transactions: pd.DataFrame) -> pd.DataFrame:  
    usable = transactions.loc[  
        transactions["amount"].notna() & transactions["amount"].ge(0)  
    ].copy()  
    usable.loc[:, "month"] = (  
        pd.to_datetime(usable["transaction_date"], utc=True)  
        .dt.to_period("M")  
        .dt.to_timestamp()  
    )  
    return (  
        usable.groupby(["customer_id", "month"], as_index=False)  
        .agg(orders=("amount", "size"), revenue=("amount", "sum"))  
    )
```

```
def attach_customers(monthly: pd.DataFrame, customers: pd.DataFrame) -> pd.DataFrame:  
    return monthly.merge(  
        customers,
```

```

    on="customer_id",
    how="left",
    validate="many_to_one",
)

```

`assign` is a useful analogue to `mutate` when its lambdas remain short. `query` can read well in controlled code, but ordinary boolean masks are clearer when column names are dynamic or null semantics are important. Use `merge` instead of a home-grown lookup; it has the cardinality checks you want. When a transform becomes difficult to read, stop chaining and name the intermediate state.

2.2 A compact translation table

R habit	Python baseline	Migration guardrail
<code>filter()</code>	boolean mask plus <code>.loc[...]</code>	use <code>.copy()</code> before later assignment
<code>mutate()</code>	<code>.assign()</code> or <code>.loc[:, col] = ...</code>	make the output column and null rule visible
<code>summarise()</code>	<code>groupby(..., as_index=False).agg(...)</code>	keep group keys as columns
<code>left_join()</code>	<code>merge(..., how="left", validate=...)</code>	declare expected cardinality
<code>case_when()</code>	<code>numpy.select()</code>	test condition ordering and default
<code>pivot_*()</code>	<code>pivot</code> , <code>pivot_table</code> , <code>melt</code>	check duplicate key behaviour
<code>lag()</code> / rolling	<code>groupby().shift()</code> / <code>rolling()</code>	sort within each group first

Here is a `case_when`-style rule with a testable default. The order is significant: the first true condition wins.

```

import numpy as np

def add_risk_band(frame: pd.DataFrame) -> pd.DataFrame:
    conditions = [
        frame["churn_risk"].ge(0.80),
        frame["churn_risk"].ge(0.50),
        frame["churn_risk"].ge(0.20),
    ]
    return frame.assign(
        risk_band=np.select(

```

```

        conditions,
        ["critical", "high", "moderate"],
        default="low",
    )
)

```

For grouped lags and windows, ordering is part of the contract. A groupby does not sort time for you.

```

def add_history_features(frame: pd.DataFrame) -> pd.DataFrame:
    out = frame.sort_values(["customer_id", "month"]).copy()
    group = out.groupby("customer_id", sort=False)["revenue"]
    out.loc[:, "revenue_lag_1"] = group.shift(1)
    out.loc[:, "revenue_roll_3"] = (
        group.rolling(3, min_periods=3).mean().reset_index(level=0, drop=True)
    )
    return out

```

2.3 Test the properties that make an analytical result trustworthy

Do not merely compare a final CSV. Test the contracts close to the transformation that owns them. A small fixture with awkward values is often more useful than a large production extract.

```

import pandas as pd
from pandas.testing import assert_frame_equal

from scorecard.transforms import build_monthly_sales

def test_monthly_sales_discards_invalid_amounts() -> None:
    transactions = pd.DataFrame({
        "customer_id": [1, 1, 1],
        "transaction_date": ["2026-01-02", "2026-01-03", "2026-01-04"],
        "amount": [10.0, None, -5.0],
    })
    actual = build_monthly_sales(transactions)
    expected = pd.DataFrame({
        "customer_id": [1],
        "month": [pd.Timestamp("2026-01-01")],
        "orders": [1],
        "revenue": [10.0],
    })
    assert_frame_equal(actual, expected, check_dtype=False)

```

That one test documents three choices: null amounts are excluded, negative amounts are excluded, and the output grain is customer-month. Those choices are more valuable than a clever expression.

3 Keep functional clarity, but use Python's native tools

purrr users already value small functions, explicit inputs, and composition. Keep that instinct. The change is that Python does not centre one package around

a single mapping grammar: you combine ordinary functions, comprehensions, iterators, exceptions, context managers, and standard-library modules.

3.1 Prefer straightforward functions and comprehensions

For simple local mapping, a comprehension is usually clearer than `map`. For a named business operation, write a named function. Both are easier to inspect and test than a large anonymous lambda.

```
def normalise_name(value: str) -> str:
    return " ".join(value.strip().title().split())
```

```
names = [normalise_name(value) for value in raw_names if value is not None]
```

Use `map` when it improves composition or when an iterator is intentional; convert it to a list only when you need materialised results. Avoid using `apply(axis=1)` as a general substitute for `dplyr` row-wise work. It is often slow, hides column dependencies, and makes it easy to miss vectorised operations. First ask whether the rule can be expressed with a vectorised column operation, `numpy.select`, `groupby`, or a merge.

3.2 Make failure policy part of the function boundary

`purrr::possibly()` and `safely()` are valuable because they make error policy explicit. Do the same in Python. Do not catch `Exception` and silently produce a plausible incomplete answer.

```
from dataclasses import dataclass
from pathlib import Path
```

```
@dataclass(frozen=True)
class ReadResult:
    path: Path
    rows: int | None
    error: str | None
```

```
def read_partition(path: Path) -> ReadResult:
    try:
        rows = len(pd.read_parquet(path))
    except (OSError, ValueError) as exc:
        return ReadResult(path=path, rows=None, error=str(exc))
    return ReadResult(path=path, rows=rows, error=None)
```

The caller now has a reviewable choice: fail the complete run when any partition fails, retry a defined transient class, or publish a clearly marked partial result. It is not acceptable to decide that accidentally in a loop.

3.3 Treat resources and configuration as explicit dependencies

Python context managers make ownership of files, database connections, and temporary resources visible. `pathlib.Path` avoids a surprising amount of path-string fragility. Pass settings into functions rather than looking them up from ambient globals.

```
from pathlib import Path
```

```
def write_report(frame: pd.DataFrame, output_dir: Path) -> Path:
    output_dir.mkdir(parents=True, exist_ok=True)
    destination = output_dir / "monthly_scorecard.parquet"
    frame.to_parquet(destination, index=False)
    return destination
```

This is the migration-friendly architecture: I/O at the edge, pure transformations in the middle, and a thin orchestrator that combines them. It creates test seams without demanding a large framework.

4 Separate inference, prediction, and visual communication

4.1 Choose a model stack by the question, not by package loyalty

R can feel like one modelling culture even when packages differ. Python makes a useful split explicit.

Question	Primary Python route	What to preserve from R
What is the estimated effect and uncertainty?	<code>statsmodels</code> formulas and diagnostics	specification, contrasts, standard errors, interpretation
Which workflow predicts best on future data?	<code>scikit-learn</code> pipeline and validation	split policy, preprocessing, metrics, threshold policy
Do we need a specialised model?	a focused library after a parity baseline	the domain assumptions, not just the estimator name

For an inference-first port, formula notation is a helpful bridge. Formula convenience is not a substitute for checking how categorical levels, missing rows, transformations, and reference categories are handled.

```
import statsmodels.formula.api as smf

fit = smf.ols(
    "revenue ~ recency_days + C(segment) + C(region)",
    data=train,
).fit(cov_type="HC3")
print(fit.summary())
```

For prediction, put preprocessing and the estimator in one pipeline. This prevents a common migration bug: fitting encoders or imputers separately on test

```
data.
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric = ["recency_days", "orders", "revenue"]
categorical = ["segment", "region"]

preprocess = ColumnTransformer([
    ("numeric", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]), numeric),
    ("categorical", Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("encode", OneHotEncoder(handle_unknown="ignore")),
    ]), categorical),
])

model = Pipeline([
    ("preprocess", preprocess),
    ("classifier", LogisticRegression(max_iter=1_000)),
])
model.fit(train[numeric + categorical], train["churned"])
```

Keep the split policy outside the estimator and make it date-aware when the R workflow was date-aware. Random folds on temporally ordered data can create leakage and make a port look better than either production system.

```
from sklearn.model_selection import TimeSeriesSplit

cv = TimeSeriesSplit(n_splits=5)
```

Validate the decisions the workflow supports, not only a familiar summary statistic. For a churn score, compare calibration, segment performance, and the threshold that determines how many customers get an intervention. AUC alone cannot tell you whether a chosen operational threshold is safe.

4.2 Rebuild chart meaning before chart styling

Do not ask which Python package is “the ggplot replacement” until you can say what the chart must communicate. seaborn plus matplotlib is a durable

default for static reporting. plotnine can be useful when grammar continuity reduces transition risk. The important test is semantic parity: same measure, filter, group, scale, timezone, annotation, and treatment of missing values.

```
import matplotlib.pyplot as plt
import seaborn as sns

def plot_monthly_revenue(frame: pd.DataFrame) -> plt.Figure:
    fig, ax = plt.subplots(figsize=(9, 4.5))
    sns.lineplot(data=frame, x="month", y="revenue", hue="segment", ax=ax)
    ax.set(
        title="Monthly revenue by customer segment",
        xlabel="Month",
        ylabel="Revenue",
    )
    ax.legend(title="Segment")
    fig.tight_layout()
    return fig
```

Use a small plotting helper module to own colours, fonts, labels, export dimensions, and accessibility checks. A team should not rebuild the corporate chart style in every script. For critical reports, save the R and Python charts side by side and review the underlying summary table as well as the pixels.

5 Improve performance in the order that preserves trust

Performance work is a decision sequence, not a race to add workers. Start with a representative sequential run, record elapsed time and peak memory, then identify the expensive stage. In table workflows, the largest gains often come from reducing I/O, selecting fewer columns, fixing types, avoiding Python-level row loops, and vectorising an operation. Parallelism comes later.

```
from time import perf_counter

started = perf_counter()
result = build_monthly_sales(transactions)
elapsed_seconds = perf_counter() - started
print(f"monthly_sales_seconds={elapsed_seconds:.2f} rows={len(result)}")
```

This is not a substitute for a profiler, but it establishes a reproducible before/after boundary. Keep benchmark inputs and environment conditions fixed. Otherwise, an apparently faster port may simply have read a warmer cache or processed less data.

5.1 Vectorise and reduce data movement first

Avoid `DataFrame.apply(axis=1)` when a column operation, group operation, merge, or numpy expression can represent the same rule. Read columns rather than entire files when possible. Prefer Parquet for repeated analytical pipelines when you control both ends. If one process runs out of memory, fix the data shape or process in partitions before assuming more processes will help; each process can multiply the memory pressure.

`polars` is a reasonable escalation when measured table work is the bottleneck and its execution model suits the team. Its introduction should be a separately validated change, not something silently mixed into a parity port.

5.2 Add parallelism only for independent, coarse work

The Global Interpreter Lock (GIL) means threads do not generally run Python bytecode in parallel. Threads remain useful for I/O-bound work, where the process is mostly waiting for remote services or files. For independent CPU-bound Python tasks, a process pool is the standard-library baseline. Native numerical libraries may already release the GIL or use their own threads, so measure before adding another layer of concurrency.

Workload	First choice	Why	Main failure mode
Vectorised dataframe calculation	one process	low overhead and easy debugging	an accidental Python row loop
Remote/API/file waiting	<code>ThreadPoolExecutor</code>	waiting overlaps	rate limits, retry storms, partial results
Independent CPU-heavy tasks	<code>ProcessPoolExecutor</code>	separate interpreter processes	serialising large objects and memory duplication
Larger table/execution graph	evaluate Dask or another platform	scheduler can manage partitions	adding infrastructure before proving need

Pass small, serialisable task descriptions such as file paths and configuration, rather than a large live dataframe. Keep worker functions at module top level; nested functions and notebook state cause portability problems, especially on platforms that use the `spawn` start method.

```
from concurrent.futures import ProcessPoolExecutor
from pathlib import Path
```

```
def score_partition(path: Path) -> tuple[Path, int]:
    partition = pd.read_parquet(path)
    # CPU-heavy scoring that uses only this partition's data.
    return path, len(partition)
```

```
def score_all(paths: list[Path], workers: int = 4) -> list[tuple[Path, int]]:
    with ProcessPoolExecutor(max_workers=workers) as pool:
        return list(pool.map(score_partition, paths))
```

Give concurrent code cancellation, timeout, retry, and partial-publication rules. A Future that fails is useful evidence; swallowing it is not. Keep the sequential implementation available as a correctness baseline until the concurrent path has been exercised under realistic failure conditions.

6 One end-to-end case: migrate a monthly churn scorecard

Consider a trusted R workflow that reads transactions and customer attributes, produces a monthly customer feature table, fits a churn model, publishes a segment chart, and hands a priority list to a retention team. Its migration goal is not “use Python everywhere.” Its goal is to produce the same defensible intervention decisions with a workflow the team can test and operate.

6.1 Stage 1: freeze the reference outcome

Choose a completed reporting month. Preserve raw input files or immutable references, the R feature table, scored output, chart data, model specification, and the priority-list threshold. Write acceptance criteria such as:

Output	Acceptance rule
Customer-month table	same keys; row count and aggregates within agreed tolerance

Output	Acceptance rule
Features	null rate and distribution match expected policy
Model	same temporal split; comparable calibration and segment metrics
Priority list	agreement around the action threshold is investigated, not waved away
Chart	same measure, segments, time range, scale, and title meaning
Run	completes within the agreed resource envelope and leaves an audit trail

The priority list deserves special care. Two scores can be numerically close but fall on opposite sides of an operational threshold. Review disagreements around that boundary row by row; they often reveal a join, null, encoding, or date-cut defect.

6.2 Stage 2: port transformations before modelling

Build a Python feature function that accepts dataframes and returns a dataframe.

Validate its grain and invariants before fitting any model.

```
def build_churn_features(
    transactions: pd.DataFrame,
    customers: pd.DataFrame,
    as_of: pd.Timestamp,
) -> pd.DataFrame:
    txns = transactions.loc[
        pd.to_datetime(transactions["transaction_date"], utc=True).le(as_of)
        & transactions["amount"].ge(0)
    ].copy()
    txns.loc[:, "transaction_date"] = pd.to_datetime(txns["transaction_date"], utc=True)

    monthly = (
        txns.groupby("customer_id", as_index=False)
        .agg(
            orders=("amount", "size"),
            revenue=("amount", "sum"),
            last_transaction=("transaction_date", "max"),
        )
    )
    monthly.loc[:, "recency_days"] = (
        as_of - monthly["last_transaction"]
    ).dt.days
    return monthly.merge(
```

```

customers[["customer_id", "segment", "region"]],
on="customer_id",
how="left",
validate="one_to_one",
)

```

Test the edge cases that caused trouble in the original workflow: customers with no transactions, refunds, duplicate dimension records, null segments, and timestamps at the cut-off. This is the right point to discover a legacy ambiguity. If the R workflow's behaviour is unclear, record a decision rather than reverse-engineering it silently.

6.3 Stage 3: compare the model and chart at decision level

Fit the initial Python baseline on the same temporal split. For analyst review, a statsmodels formula may be the fastest interpretable checkpoint. For operational scoring, use the scikit-learn pipeline from Section 4. Do not treat them as competing implementations if the programme needs both: one can preserve familiar inference artefacts while the other controls preprocessing and scoring.

Generate an explicit comparison report:

```

comparison = r_scores.merge(
    python_scores,
    on="customer_id",
    suffixes=("_r", "_python"),
    validate="one_to_one",
)
comparison.loc[:, "score_delta"] = (
    comparison["score_python"] - comparison["score_r"]
)
comparison.loc[:, "priority_disagreement"] = (
    comparison["score_r"].ge(0.65) != comparison["score_python"].ge(0.65)
)

```

Investigate the largest deltas and every threshold disagreement before debating coefficients. Then compare the chart's input table, labels, scale, and ordering. A visually similar chart built from a different feature grain is a defect, not a successful port.

6.4 Stage 4: profile, package, and make the cutover reversible

Run the complete sequential Python job on representative data. If it meets the agreed service level, stop. Package it with pinned dependencies, tests, run meta-data, and a documented invocation. If it misses, profile the slow stage and remove the measured cause. Only use a process pool when independent scoring partitions remain the real bottleneck after vectorisation and I/O work.

For the initial cutover, dual-run R and Python for a fixed, agreed period. Publish the Python result only after the comparison checks pass; retain the R result as rollback until the owner accepts the new path. Dual-running is temporary evidence, not a permanent architecture.

7 Migrate as a team, with a recovery path

7.1 Put shared agreements in the repository

Migration quality falls when each person selects a different dataframe style, null policy, model split, plotting default, and approach to errors. Establish a small set of recorded agreements early:

- business keys stay explicit columns and important joins declare cardinality;
- stable transformations live in modules, not only notebooks;
- every port names its R reference artefacts and parity tolerances;
- prediction pipelines fit preprocessing only on training data;
- chart helpers own shared styling and export settings;
- performance claims include a reproducible before/after measurement;
- a production change names its owner, monitoring signal, and rollback procedure.

This is not bureaucracy. It lets an R expert review the domain contract while a Python-oriented colleague reviews runtime and packaging, without either having to infer the other person's assumptions from syntax.

Use pull requests as migration evidence. A good migration PR contains the smallest coherent workflow change, parity tests or a comparison artefact, representative output review, and an explanation of any intentional difference from R. Do not hide a new library, data-grain change, and performance rewrite in the same PR; then a failure has no obvious cause.

7.2 Recognise the recurring anti-patterns

Anti-pattern	Why it fails	Recovery
Literal translation of every pipe	preserves surface syntax but hides Python data and state rules	reframe as named transformations with contracts

Anti-pattern	Why it fails	Recovery
One notebook becomes the application	hard to test, rerun, and review	move stable logic into modules; keep notebooks for exploration
<code>apply(axis=1)</code> everywhere	slow row-wise Python disguises a vectorisation opportunity	replace with column operations, grouping, joins, or a measured loop
Parallelise before profiling	creates harder failures without proving a gain	restore sequential baseline; measure, then change one bottleneck
Compare only final accuracy	misses leakage, calibration, thresholds, and subgroup failures	compare data, features, splits, metrics, and action boundary
“The plot looks right”	appearance can mask a different measure or filter	compare the chart input table and semantic contract
Replace R everywhere at once	destroys the trustworthy reference and overwhelms review	migrate one workflow, dual-run, then repeat

7.3 A concise recovery sequence

If a migration has already become a tangle of notebooks, package experiments, and unexplained output differences, stop adding features. Select one decision-critical workflow and reset it:

1. Freeze a representative R input and output.
2. State the data, model, visual, and operational acceptance criteria.
3. Rebuild only the transformations as pure Python functions and test their edge cases.
4. Reintroduce the model and chart, investigating disagreements near the action threshold.
5. Profile the sequential complete run.
6. Add an execution optimisation only when a measured bottleneck justifies it.

7. Dual-run for a bounded period, document ownership and rollback, then cut over.

The transition succeeds when a team can explain, test, run, and change the Python workflow with at least as much confidence as the R version. That is a much better finish line than matching every line of syntax.

7.4 Further Reading

- [pandas user guide](#) for dataframe semantics and missing-data behaviour.
- [Python standard library: concurrent.futures](#) for thread and process executors.
- [scikit-learn user guide](#) for pipelines, preprocessing, and model validation.
- [statsmodels documentation](#) for inference-oriented models and diagnostics.
- [matplotlib documentation](#) for plotting primitives and export control.